Mocha also has the with method to specify arguments to a mock or a stub, as in Project.expects(:find).with(1). There are also some more advanced but occasionally useful things you can do with with. First off, with can take a block as an argument, in which case the value passed to the method is passed on to the block. If the block returns true, then the expectation is considered matched:

```
proj = Project.new()
proj.expects(:status).with { |value| value % 2 == 0 }
    .returns("Active")
proj.expects(:status).with { |value| value % 3 == 0 }
    .returns("Asleep")
```

If more than one block returns true, the last one declared wins. If none of the blocks return true, you get the same unexpected invocation error as in the preceding code.

The argument to with can also be one of a series of parameter matchers. Probably the most useful one is instance_of, which you can use to simulate behavior given different types of input:

```
proj = Project.new()
proj.expects(:tasks_before).with(instance_of(Date)).returns(3)
proj.expects(:tasks_before).with(instance_of(String)).raises(Exception)
```

The instance_of matcher or any other Mocha matcher can be negated with the Not method. (Yes, it's capitalized, presumably to avoid weird parse collisions with the keyword not.)

```
proj = Project.new()
proj.expects(:tasks_before).with(Not(instance_of(Date))).returns(3)
```

The only other such specialized matcher I've ever found remotely valuable in practice is has_entry, which works against a hash:

```
proj.expects(:options).with(has_entry(verbose: true))
```

The stub in this snippet will match any hash argument that contains a verbose: true entry, no matter what the other contents of the hash might be. Using the has_entry matcher is occasionally valuable against, say, an ActiveRecord or controller method that expects a hash where you care about only one of the methods.

There's about a dozen more of these matchers, many of which are, shall I say, somewhat lacking in real-world value. Rather than cluttering your head with a bunch of stuff you'll never use, I invite you to check out the Mocha docs online for a full listing if you're interested.[4]

4. http://gofreerange.com/mocha/docs